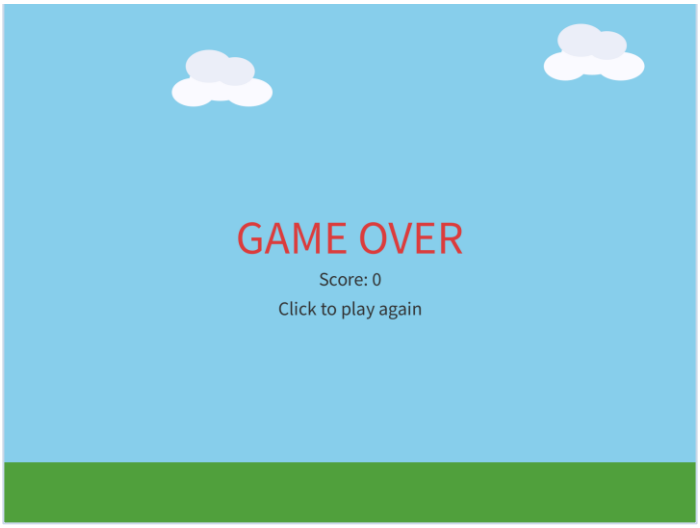
Chapter 7 - Build Your Game

Everything you have learned, in one game that is yours. This is the one you show people.

Restarting a game sounds hard. You just start it again.




```
void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}
```

Chapter 6 ended with a game that can end. Once it's over, though, the only way to play again is to press Stop and Run.

Today you finish it. A way to win as well as lose, a click to play again, and then the rest of the time to make it a game somebody else wants to play.

There's exactly **one** new idea in this chapter: restarting. Everything else you've already built.

This chapter takes three meetings, and does it in four moves.

Part	The idea
One	**You already know all of this.** Read the starter, and find every piece.
Two	**One full loop.** Win, lose, click, and play again, before you change anything.
Three	**Make it yours.** Your character, your world, your rules.
Four	**Play-test, and show it.** Somebody else plays your game.

Parts One and Two are the first meeting. Part Three is the second. Part Four is the third.

Part One - You already know all of this

7.1 One sentence first

Your turn. Before any code, write one sentence describing your game. Who's your character? What are they collecting, and what are they avoiding? Two minutes.

If you can say what your game is, you'll build a much better one.

7.2 The starter

You can start from your Chapter 6 sketch, or from the starter below, whichever is closer to what you want. Either way, keep your own `MyCharacter` and `Cloud` tabs.

Build it. Paste this into your main tab:

```
// Lesson 7 - Build Your Game.  Replace everything marked CHANGE with your own
choices!
import coxprogramming.processing.sprites.*;

MyCharacter player;
Cloud cloud1;
Cloud cloud2;
Block ground;

Blob coin1;      // things to collect
Blob coin2;
Blob coin3;
Blob hazard1;    // things to avoid
Blob hazard2;

int score = 0;
int goal = 15;               // CHANGE: the score you need to win
boolean gameOver = false;
boolean playerWon = false;

void setup() {
  size(800, 600);

  // Scenery never changes during play, so it is built once, here.
  ground = new Block(this, 0, height - 70, width, 70, color(80, 160, 60));  //
CHANGE: your ground
  cloud1 = new Cloud(this, 150, 90, 0.5);
  cloud2 = new Cloud(this, 520, 60, 0.8);

  restartGame();
}

void draw() {
  background(135, 206, 235);  // CHANGE: your sky / background color

  cloud1.move();
  cloud1.drawSprite();
  cloud2.move();
  cloud2.drawSprite();

  ground.drawSprite();
```

```
  if (!gameOver && !playerWon) {
    playGame();
  } else if (playerWon) {
    showWin();
  } else {
    showGameOver();
  }
}

void playGame() {
  // --- Collectibles ---
  coin1.move();
  coin1.drawSprite();
  coin2.move();
  coin2.drawSprite();
  coin3.move();
  coin3.drawSprite();

  // --- Hazards ---
  hazard1.move();
  hazard1.drawSprite();
  hazard2.move();
  hazard2.drawSprite();

  // --- Player ---
  player.followMouse();  // or keyboardControl() in keyPressed()
  player.drawSprite();
```

```
  // --- Collecting:  score a point, and the coin jumps somewhere new ---
  if (player.collidesWith(coin1)) {
    score = score + 1;
    coin1.moveTo(random(width), random(80, height - 150));
  }
  if (player.collidesWith(coin2)) {
    score = score + 1;
    coin2.moveTo(random(width), random(80, height - 150));
  }
  if (player.collidesWith(coin3)) {
    score = score + 1;
    coin3.moveTo(random(width), random(80, height - 150));
  }

  // --- Lose condition ---
  if (player.collidesWith(hazard1)) {
    gameOver = true;
  }
  if (player.collidesWith(hazard2)) {
    gameOver = true;
  }

  // --- Win condition ---
  if (score >= goal) {
    playerWon = true;
  }

  // --- Score display ---
  fill(50);
  textSize(22);
  text("Score: " + score + " / " + goal, 20, 32);
}
```

```
void showWin() {
  textAlign(CENTER, CENTER);
  fill(80, 200, 100);
  textSize(54);
  text("YOU WIN!", width/2, height/2 - 30);
  fill(50);
  textSize(22);
  text("Score: " + score, width/2, height/2 + 18);
  text("Click to play again", width/2, height/2 + 52);
  textAlign(LEFT, BASELINE);
}

void showGameOver() {
  textAlign(CENTER, CENTER);
  fill(220, 60, 60);
  textSize(54);
  text("GAME OVER", width/2, height/2 - 30);
  fill(50);
  textSize(22);
  text("Score: " + score, width/2, height/2 + 18);
  text("Click to play again", width/2, height/2 + 52);
  textAlign(LEFT, BASELINE);
}

void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}

void restartGame() {
  score = 0;
  gameOver = false;
  playerWon = false;

  // CHANGE: your player's starting position
  player = new MyCharacter(this, width/2, height/2);
  player.setVelocity(4, 0);
  // Collectibles - CHANGE: color, size, speed
  coin1 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));
  coin2 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));
  coin3 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));

  // Hazards - CHANGE: color, size, speed.  They start in the top corners, away
from the player.
  hazard1 = new Blob(this, 80, 80, 22, 3, 2, color(220, 60, 60));
  hazard2 = new Blob(this, width - 80, 80, 22, -2, 3, color(220, 60, 60));
}
```

Look through it. Every piece of it is something you've written before:

Piece	What it does	You met it in
`setup()`	builds the scenery once, then calls `restartGame()`	Chapter 1
`draw()`	the sky, clouds, and ground, then picks a screen	Chapters 1 and 5
`MyCharacter`, `Cloud`	your character, and your scenery	Chapters 2 and 5
`setVelocity()`, `followMouse()`	a speed, and the mouse	Chapters 3 and 4
`playGame()`	three coins, two hazards, your player, and the score	Chapter 6
`showWin()`, `showGameOver()`	the two end screens	Chapter 6
`mousePressed()`	a click	Chapter 4
`restartGame()`	**new**	today

The coins and hazards are plain colored blobs. They're placeholders, there so the code runs. Replacing them with things that belong in your world is the assignment.

If you drive with the arrow keys, put your `keyPressed()` from Chapter 4 at the bottom, and change `player.followMouse();` in `playGame()` to `player.move();`.

7.3 Three modes

Chapter 6 had two modes: playing, and game over. This game has three, because winning and losing should look different. `playerWon` joins `gameOver`, and `draw()` picks one of three screens:

```
  if (!gameOver && !playerWon) {
    playGame();
  } else if (playerWon) {
    showWin();
  } else {
    showGameOver();
  }
```

> **Aside: `&&` and `else if`**
>
> `&&` means **and**. `!gameOver && !playerWon` reads *not lost, and not won*: still playing. `else if` adds another question to an `if`: if the first one was false, try this one. Processing goes down the list, and the first one that's true wins. The last `else` catches whatever's left.

You win when `score >= goal`. The `goal` is at the top, `15` to start. Change it to make the game longer or shorter.

7.4 Restarting

To restart a game, you might think you need to undo everything that happened. That's much harder than it sounds, and it's easy to miss something.

You just run the starting steps again. Make a new player, make new coins and hazards, set the score back to `0`, and set the modes back to `false`. That's `restartGame()`:

```
void restartGame() {
  score = 0;
  gameOver = false;
  playerWon = false;

  player = new MyCharacter(this, width/2, height/2);
  // ... brand-new coins, and brand-new hazards
}
```

Look at `setup()`. It builds the ground and the clouds, and then it calls `restartGame()`. Starting the game and restarting it are the same thing, so they're the same code. The ground and clouds are built once, in `setup()` alone, because nothing in the game ever changes them.

The restart happens on a click:

```
void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}
```

Aside: ||

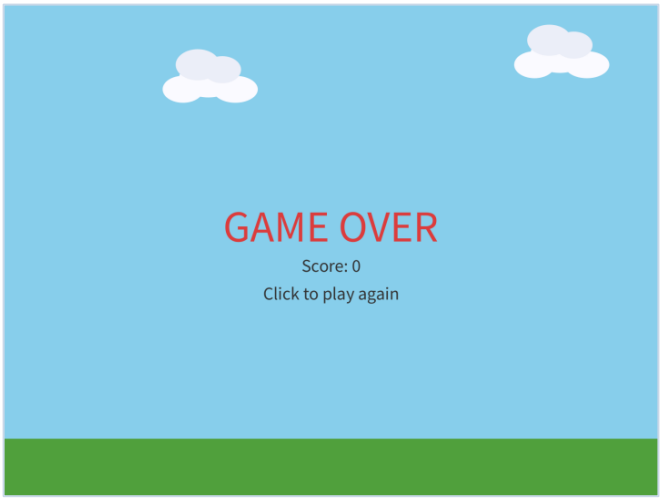
`||` means **or**. `gameOver || playerWon` is *we lost, or we won*. The `if` guards the restart, so a click while you're playing leaves your game alone.

Part Two - One full loop

Step 1 - Lose, and play again

Build it. Run the starter. Play it until you lose.

> **Check.** *GAME OVER*, your score, and *Click to play again*.



Lost.

Now click.

> **Check.** A fresh game: the score is `0`, your player is back in the middle, and the hazards are back in the top corners.



Ten frames after the click.

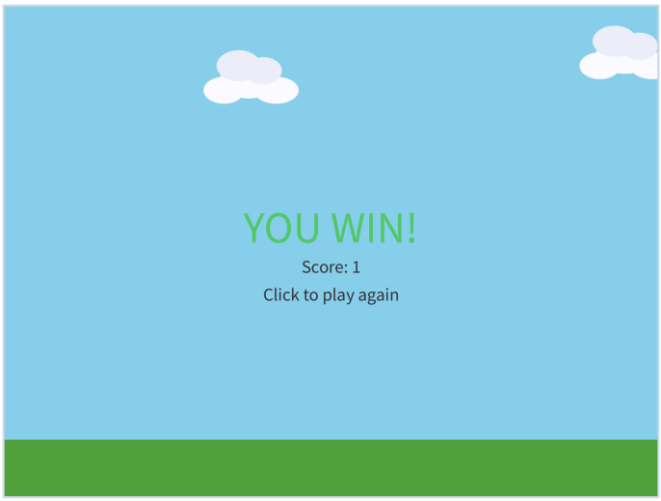
Make sure that works before you change anything.

Step 2 - Win it, by cheating

Winning takes 15 coins, which is a lot of playing just to check one screen. So test it by cheating, the way you did in Chapter 6.

Build it. Change `int goal = 15;` to `int goal = 1;`, and run. Touch one coin.

Check. *YOU WIN!*



One coin, and a win.

Click to play again, and make sure the win restarts too. Then set the goal back.

Watch out. `restartGame()` has to put back **every** variable that the game changes. Leave one out, and the restart misses it. Take out `gameOver = false;`, lose, and click: the game rebuilds everything else, and you're still on the game over screen. Check every variable at the top of your sketch against `restartGame()`, one at a time.

Step 3 - One full loop, in your world

Build it. Bring in your own world from Chapter 5: your ground, your background, your decorations. Then replace the first placeholder coin with something that belongs there.

Get one full loop working first: your player, your coins, and one way to win or lose. Play it, properly, before you add anything else. The games that go wrong in this chapter are almost always the ones with five new features and no loop that plays.